Truthprint: Designing an Invariant-Constrained Semantic Intermediate Representation for Provenance Watermarking

Anonymous ACL submission

Abstract

Token-level LLM watermarks are fragile under translation and paraphrasing because their signal lives in surface tokens. We propose *Truthprint*, a semantic intermediate representation that separates *locked invariants* (polarity, quantity, time, attribution, roles) from *controlled realization* via a machine-checkable typed mutability contract. The mechanism that survives transformation is *ledger-based authenticated semantic provenance*—a keyed tag over the invariant digest, re-verified after transformation—not an in-text watermark; hiding a payload in realization freedom is instead an honest negative under real MT (a surface parser recovers 0/192 NLLB outputs). On 4,800 real NLLB translations across six languages (committed in-repo) a Stage-2 extractor recovers the invariants (0.83–0.99; closed categoricals ≥ 0.97) and the scheme attributes translated documents cross-lingually (core6 document attribution 1.000 for German/Hindi, 0.90–0.98 otherwise), rejecting single-field tampers (≥ 0.996) and *localizing* the changed field—unlike a token watermark (KGW collapses under translation), an embedding region (SemStamp degrades cross-lingually at matched FPR), or robust hashing. Benign paraphrase is authenticated (1.00); a fixed-lexicon adaptive attacker removes the mark, but open-vocabulary neural frontends (GPT-4o-mini, Llama-3.3-70B) defend even held-out attacks (ValidRemoval 0.05–0.10) at unchanged tamper rejection. We prove three *conditional* properties reducing to the MAC, and show the operative false-positive floor is the contract-collision rate, not $2^{-\tau}$. All results are closed-domain and small-scale; wide-coverage parsing, official baselines (SynthID, SWAN), and scale remain future work. Truthprint verifies consistency with a participating keyed generator, not arbitrary AI text.

Keywords. Intermediate representation, authenticated provenance, LLM watermarking, semantic provenance, translation robustness

1 Introduction

Large language models can generate fluent text at scale, creating a growing need for provenance mechanisms that survive ordinary content transformation. Generation-time watermarking modifies the decoding process so that an authorized detector can identify a statistical signature in generated text. SynthID-Text demonstrated that such watermarking can be integrated into sampling without retraining the underlying LLM and can be detected without rerunning the generator (Dathathri et al., 2024).

Regulation motivates machine-readable marking (EU AI Act Article 50 (European Parliament and Council, 2024)), useful only if robust (Liu et al., 2024c); we treat robustness as a technical objective, not a compliance claim. Provenance also aids data hygiene, since training on unattributed synthetic text degrades models over generations (Shumailov et al., 2024).

Token-level methods, however, embed their signal in surface token choices. Translation, paraphrasing, sentence restructuring, and summarization replace many of those choices while preserving substantial meaning. This creates a structural mismatch: the desired provenance claim concerns the persistence of content identity, whereas the carrier often depends on language-specific lexical decisions.

Recent semantic watermarks use sentence embeddings, contextual representations, or structured graphs (Hou et al., 2024; Ren et al., 2024; Liu et al., 2024b; Ye et al., 2026; Schäfer et al., 2026), but semantic similarity alone

does not distinguish meaning that must never change from choices that may safely vary: two sentences can be embedding-close yet differ in negation, quantity, temporal scope, attribution, modality, or causal direction.

This paper proposes *Truthprint*, a watermark-specific semantic IR that explicitly separates:

1. **semantic invariants**, which must remain fixed; and

2. **controlled realization variables**, which may encode watermark symbols only when semantic and pragmatic equivalence is validated.

The central novelty is therefore not the invention of a new surface language. It is a machine-checkable contract defining which transformations are permitted to carry a watermark and which semantic fields are locked. Figure 1 shows the end-to-end data flow.

The delta over the closest prior work is precise. Embedding-based semantic watermarks (Hou et al., 2024; Ren et al., 2024; Liu et al., 2024b; Schäfer et al., 2026) define admissibility through a *metric* (closeness under a learned distance); Truthprint defines it through a *typed predicate*—admissible only when a symbolic invariant relation holds field by field—so a watermark symbol ranges over key-dependent invariant-equivalence classes rather than tokens or embedding regions (Sec. 9). Conditional on sound extraction, encoding stays within a class and preserves protected fields, and because attribution is gated by a payload bound to the invariant digest, fresh-payload forgery reduces to MAC security. SWAN (Ye et al., 2026) shares a graph carrier but adopts an existing formalism wholesale, whereas Truthprint types each field with an explicit mutability policy validated before release.

1.1 Contributions

- **Method.** We formulate watermark carriers as secret-keyed realization choices constrained by a field-level mutability contract (*locked*/*conditional*/*realization-only*), combining uncertain-carrier erasures, error correction, and invariant-bound payload authentication.

- **Conditional analysis.** Assuming sound invariant extraction and validation we establish protected-field fidelity; under standard cryptographic assumptions fresh-payload forgery and per-test tag acceptance reduce to the MAC; and we show the operative false-positive floor is the contract-collision rate, with a measured crossover to the $2^{-\tau}$ bound.

- **Reproducible real-MT evidence.** On 4,800 committed NLLB translations across six languages the meaning layer survives and authenticates cross-lingually where surface carriers and a real token watermark do not; at a matched FPR it beats a real semantic baseline and robust hashing and alone rejects *and* localizes single-field tampers; and open-vocabulary neural frontends defend held-out adaptive attacks. Every number reproduces in-repo with no GPU; wide-coverage parsing and scale remain open.

Claim–evidence boundary. Table 7 (appendix) separates what the artifact *demonstrates*, what is proved only under explicit assumptions, and what remains a hypothesis (e.g., "language-neutral" names the intended schema interface, not a measured property).

2 Background and Related Work

Text watermarking is motivated by transparency regulation—Article 50 of the EU AI Act requires machine-readable marking of synthetic text (European Parliament and Council, 2024), useful only if *robust* to paraphrase and translation (Liu et al., 2024c)—though no watermark alone establishes compliance.

Token-level and provable watermarks. The green–red list watermark (Kirchenbauer et al., 2023) and SynthID-Text (Dathathri et al., 2024; Google DeepMind, 2024; Google, 2026), which generalizes it via tournament sampling, embed a keyed statistic in token selection; analysis shows detector robustness depends on its statistical assumptions (Omidi et al., 2026). A parallel line seeks formal guarantees: provable robustness for a unigram watermark (Zhao et al., 2024), distortion-free editing-robust watermarks (Kuditipudi et al., 2024), cryptographically undetectable watermarks (Christ et al., 2024), publicly verifiable unforgeable watermarks (Liu et al., 2024a),

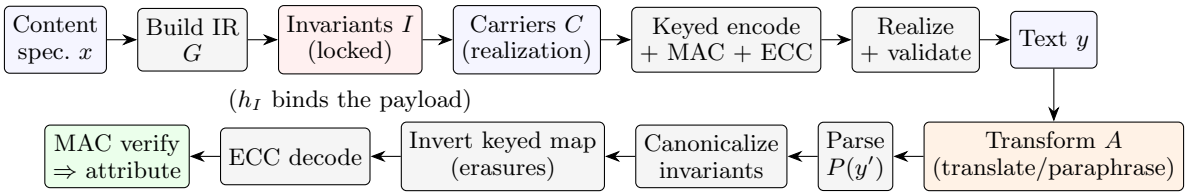


Figure 1: Truthprint data flow. Encoding (top, left→right) locks the invariant set I, maps an authenticated, error-corrected payload onto realization carriers C, and validates every candidate against the contract before emitting text. Detection (bottom, right→left) re-parses possibly transformed text, canonicalizes invariants, inverts the keyed map with erasures, decodes, and attributes only on MAC verification. The invariant digest h_I binds the payload, so an invariant-altering transform breaks attribution (Theorem 2).

and low-FPR statistical tests (Fernandez et al., 2023). All carry the signal in token decisions, which meaning-preserving transformations replace; He et al. show token watermarks lack cross-lingual consistency and are removed by pivot translation (He et al., 2024). Truthprint inherits the cryptographic spirit at the *provenance* layer—fresh-payload unforgeability and per-candidate tag acceptance (Theorems 1, 2) reduce to MAC security—with no claim of undetectability.

Semantic and structured watermarks. Embedding-region and semantic-context methods—SemStamp (Hou et al., 2024), SemaMark (Ren et al., 2024), semantic-invariant watermarking (Liu et al., 2024b), dual embeddings (Schäfer et al., 2026)—reduce token dependence; SWAN carries the signal in AMR structure (Ye et al., 2026), and SynGuard hybridizes token masking with semantic alignment (Han et al., 2025). But embedding geometry does not expose *which* typed fact changed: high similarity can coexist with altered polarity, quantity, attribution, modality, or causal direction. Truthprint instead defines a watermark-specific typed IR, assigns each field a mutability policy, and validates every realization against an immutable contract before release.

Attacks and robust hashing. Watermark presence and family can be identified by black-box probing (Gloaguen et al., 2025), motivating a Kerckhoffs threat model (security in the key, not obscurity); scrubbing and spoofing under distillation (Yi et al., 2025) and radioactive traces in fine-tuned models (Sander et al., 2024) motivate MAC-authenticated payloads rather than statistical agreement; and detector-guided adversarial paraphrasing (Cheng et al., 2025) motivates counting a removal as successful only when it both evades detection and preserves meaning:

$$\text{ValidRemoval} = \mathbf{1}\big[D_K(y') < \tau_D\big] \cdot \mathbf{1}\big[\text{InvariantEq}(P(y), P(y')) = 1\big]. \quad (1)$$

Registering a meaning-digest tag and re-verifying it after transformation resembles *robust hashing* (MinHash (Broder, 1997), SimHash (Charikar, 2002), learned semantic hashing), but those answer a *similarity* question that cannot localize a typed field change—a single-field edit stays within tolerance (Sec. 10.5). Truthprint instead hashes a *typed invariant contract* with a keyed MAC, a typed, keyed member of that family whose false-positive floor is the analyzed contract-collision probability (Sec. 10.4). Table 8 (appendix) positions it against representative systems.

3 Problem Definition

For a content specification x, secret key K, payload m, encoder E_K, language-ℓ realizer R_ℓ, transformation A, multilingual parser P, and detector D_K, the generated text is $y = R_\ell(E_K(x, m))$, an observed transform is $y' = A(y)$, and detection returns $(s, \hat{m}, c) = D_K(P(y'))$ (score, recovered payload or prefix, confidence report). The design targets four objectives: (i) *semantic fidelity* $I(P(y)) = I(x)$ for invariant extractor I; (ii) *natural-language quality* above deployment thresholds; (iii) *robust detection* under transformations that preserve sufficient invariant and carrier evidence; and (iv) *controlled authenticated false positives*—under the stated PRF and independence assumptions an independently produced decoded payload rarely passes the keyed tag

check, with the statistical fallback separately calibrated.

4 Threat Model

Benign transformations. The detector should tolerate Unicode normalization, punctuation changes, spelling correction, translation, round-trip translation, synonym substitution, active–passive alternation, clause splitting and merging, sentence-level paraphrasing, moderate paragraph rewriting, and limited insertion or deletion.

Attackers. A non-adaptive attacker applies off-the-shelf translation, paraphrasing, or summarization without knowing K. An adaptive attacker may know the schema and detector design, repeatedly rewrite text, splice documents, target known carrier families, or query an exposed detector.

Out of scope. Truthprint does not identify all AI-generated text. It verifies consistency with a participating keyed generator. Absence of a signal does not prove human authorship.

5 Truthprint Intermediate Representation

5.1 Three-Layer Structure

A Truthprint document contains three conceptual layers.

Layer A: Semantic invariants. Entities, predicates, semantic roles, polarity, negation scope, modality, quantities, temporal intervals, causal direction, conditions, exceptions, attribution, quotation status, and uncertainty.

Layer B: Controlled realization variables. Voice, event/state framing, clause packaging, discourse-connective family, proposition ordering, nominal/verbal realization, anaphoric choice, and optional explanatory apposition.

Layer C: Provenance and coding metadata. Scheme identifier, key identifier, payload, nonce, error-correcting-code parameters, carrier assignments, reliability estimates, invariant digest, and optional signed server-side provenance.

5.2 Typed Mutability

Each field f is assigned $\mathrm{Mutability}(f) \in \{\mathrm{locked}, \mathrm{conditional}, \mathrm{realization\text{-}only}\}$: locked fields never change, conditional fields change only when context-specific validators approve, and realization-only fields may carry watermark symbols.

5.3 Example Representation

A simplified representation (locked predicate/roles/polarity/time with a realization space over voice and event framing) is given in Appendix I.

6 Formal Semantic Contract

A Truthprint graph is $G = (V_E, V_V, E_R, E_D, \Phi)$—entity nodes, event/state nodes, semantic-role edges, discourse/logical relations, and scoped features such as negation, modality, time, quantity, and attribution—partitioned as $G = I \cup C \cup U$ into the invariant set I, the controlled carrier set C, and unconstrained style U. A transformed structure G' is admissible only when $\mathrm{InvariantEq}(I(G), I(G')) = 1$ and $Q(R_\ell(G')) \geq \tau_Q$ for a quality and contextual-consistency function Q. Hard invariants are validated symbolically where possible; soft invariants such as emphasis or pragmatic implication require bidirectional entailment, contradiction checks, and human-calibrated thresholds.

7 Watermark Encoding

The raw payload may carry a generator-family, deployment, coarse time-bucket, nonce, and policy identifier—never personal identifiers, prompts, or exact timestamps. The authenticated payload is $p = m \parallel \mathrm{MAC}_K(m \parallel n \parallel h_I)$ for nonce n and canonical invariant digest h_I, error-corrected as $c = \mathrm{ECC}(p)$ following prior robust multi-bit watermarking (Qu et al., 2025). The encoder discovers admissible carrier families per discourse unit (voice, clause packaging, discourse-connective family, nominal/verbal realization, proposition order), rejecting context-inappropriate ones such as a passive that obscures an important agent. For carrier j with alternatives $A_j = \{a_{j,0}, \ldots, a_{j,r-1}\}$, a keyed permutation $\pi_j = \mathrm{PRP}_K(\mathrm{CanonicalHash}(I, G, j, n))$ selects $a_j^\star = \pi_j(c_j \bmod |A_j|)$; because the map depends on the invariant graph and nonce, no global rule such as "passive means 1" exists. Each realized candidate is reparsed and checked

for hard-invariant equality, bidirectional entailment, numerical and entity consistency, fluency, and expected carrier recoverability; if none is valid the carrier is recorded as an erasure rather than forcing degraded text. Encoding and detection pseudocode are in Appendix I.

8 Watermark Detection

Detection segments the document, identifies language, parses each segment into Truthprint, resolves entities, normalizes predicates and roles, canonicalizes quantities and time, and extracts observable carrier alternatives. For an observed carrier a_j^{obs} it reconstructs the keyed permutation and derives $\hat{c}_j = \pi_j^{-1}(a_j^{obs})$, treating uncertain observations as erasures; an ECC decoder reconstructs candidate payloads, accepted only when their MAC verifies. When complete recovery is impossible, a calibrated fallback over n reliable binary carriers with k keyed matches uses

$$z = \frac{k - 0.5n}{\sqrt{0.25n}}, \tag{2}$$

whose production calibration must account for dependence among carrier observations.

9 Formal Analysis

We state what the design guarantees given sound primitives; what depends on semantic parsing is measured empirically (Sec. 10), and computational complexity is in Appendix H.

9.1 Carriers as Invariant-Equivalence Classes

Definition 1 (Invariant equivalence)**.** *Let $I(\cdot)$ be the invariant extractor. Two structures G, G' are invariant-equivalent, written $G \sim_I G'$, iff* $\mathrm{InvariantEq}(I(G), I(G')) = 1$. *For a discourse unit with invariants I, the admissible realization set is*

$$\mathcal{R}(I) = \{\, r : I(P(r)) = I \ \wedge\ Q(r) \geq \tau_Q \,\}. \tag{3}$$

A carrier alphabet A_j is a key-dependent partition of $\mathcal{R}(I)$ into classes; the encoder selects a class by symbol and the realizer emits any member. Thus the watermark ranges over the quotient $\mathcal{R}(I)/\!\sim_I$ rather than over tokens or embedding neighborhoods.

Assumption 1 (Validator soundness)**.** *The hard-invariant validator V is sound: if V accepts a candidate r for invariants I, then $I(P(r)) = I$. (It need not be complete; rejected admissible candidates only reduce capacity, not fidelity.)*

Proposition 1 (Conditional protected-field fidelity)**.** *Under Assumption 1, every committed output $y = R_\ell(E_K(x, m))$ satisfies $I(P(y)) = I(x)$.*

Proof sketch. Encoding commits a candidate only if V accepts it, and carriers with no accepted candidate are recorded as erasures rather than realized (Sec. 7). By soundness, acceptance implies invariant equality for that unit; taking the conjunction over committed units gives $I(P(y)) = I(x)$. □

Logit-biasing methods lack such an in-loop invariant check, so they can only measure quality preservation post hoc.

Empirical validator soundness. We measure the load-bearing assumption as the validator's false-accept rate on single-field tampers. On the closed domain the typed contract flags every tamper across all six fields (property C1, 1.000, CI $[0.998, 1.000]$) while retaining attribution across benign paraphrase (0.990); a decode-only ablation accepts every tamper (C2, 0.000), confirming that invariant re-extraction—not the coding layer—supplies soundness. Under real NLLB translation the Stage-2 extractor still rejects meaning-altering edits at ≥ 0.99 per condition (Sec. 10.4). Wide-coverage validator soundness remains open, so Proposition 1 stays conditional.

9.2 Unforgeability and False Positives

Theorem 1 (Fresh-payload unforgeability)**.** *Model the tag as $t = \mathrm{MAC}_K(m \,\|\, n \,\|\, h_I)$ truncated to τ bits. Any PPT adversary that causes authenticated attribution for a tuple (m, n, h_I) not previously tagged by the generator yields an EUF-CMA forgery against this truncated MAC with the same success probability (up to simulation overhead). Thus its success is bounded by the EUF-CMA advantage of the instantiated truncated MAC; guessing loss is already part of that advantage and is not added a second time.*

Proof sketch. Attribution requires a recovered payload whose tag equals $\mathrm{MAC}_K(m \,\|\, n \,\|\, h_I)$

on a triple not produced by the generator. The reduction answers authorized generation requests with its tagging oracle and, when the detector accepts a fresh tuple, outputs that tuple and recovered tag as its forgery. Replay of a previously tagged tuple is excluded by freshness and must instead be handled by deployment policy. □

Theorem 2 (Authenticated acceptance under an ideal-PRF model)**.** *Assume the truncated MAC is a pseudorandom function on a fresh input and the decoded candidate tag is independent of its output. For a fixed (key, nonce) test, an independently produced candidate payload passes the τ-bit tag check with probability $2^{-\tau}$, up to the PRF distinguishing advantage. For Q tested key–nonce pairs, the union bound is $Q2^{-\tau}$ plus that advantage.*

This bounds only a candidate that reaches tag verification: for q fresh candidates whose tags are independent of ideal-PRF outputs the union bound is $q2^{-\tau}$ (2.3×10^{-10} at τ=32), *not* an end-to-end detector FPR—parsing, candidate search, replay, carrier dependence, and threshold calibration all affect system-level error, and replayed outputs require a nonce registry. The 0/20000 observation of Sec. A is consistent with the bound (approx. 95% binomial upper bound 1.5×10^{-4}). When full recovery fails, the fallback z of Eq. (2) satisfies $\Pr[z \geq t \mid H_0] \leq e^{-t^2/2}$ under independent binary carriers, calibrated in deployment.

9.3 Capacity and Minimum Reliable Length

Model each carrier j as a q_j-ary symbol on an erasure channel with erasure probability ϵ_j, where ϵ_j is the chance that a transformation destroys or obscures that site. The expected recoverable information is

$$\mathbb{E}[\text{bits}] \approx \sum_j (1-\epsilon_j)\log_2 q_j. \tag{4}$$

With a rate-R code protecting an authenticated payload of $|p| = \mu + \tau$ bits, recovery succeeds with high probability once the surviving capacity exceeds $|p|$ with margin. For N uniform binary carriers ($q_j = 2$) with erasure rate ϵ and a systematic rate-R code ($k = RN$), this requires

$$(1-\epsilon)N \gtrsim k \iff \epsilon \lesssim 1-R. \tag{5}$$

Proposition 2 (Erasure recovery condition)**.** *For a fixed linear $[N,k]$ code, erasure decoding succeeds exactly when the generator-matrix rows corresponding to the surviving positions have rank k. Consequently at least k positions must survive, giving the necessary bound $\epsilon \leq 1-R$ for a realized erasure pattern. Whether recovery occurs near that boundary depends on the code construction and erasure distribution; the reference code's finite-length transition is measured rather than claimed as a universal threshold.*

Equation (5) gives a necessary carrier-count condition $N \geq \lceil(\mu+\tau)/((1-\epsilon)\log_2 \bar{q})\rceil$ with $R \leq 1-\epsilon$—a planning bound explaining why short or heavily summarized text falls below capacity (the finite-length transition in Appendix Table 2 is specific to the implemented code).

10 Experiments

We evaluate on a seeded, reproducible pipeline (implementation, full protocol, research questions, hypotheses, and deployment notes in the appendix); unless noted, results are closed-domain and small-scale with Wilson 95% CIs.

10.1 Surface carriers: an honest negative under real MT

Translating 4 documents of 16 templated sentences with a real system (NLLB-200-distilled-600M) into Korean/Hindi and back (192 outputs) and re-parsing each with the Stage-1 *surface* frontend recovers 0/192 (parse coverage 0.000, CI $[0.000, 0.020]$), including round-trip English. Hiding a payload in realization freedom therefore does not survive real translation; the bottleneck is wide-coverage parsing, not the coding or cryptographic core (Appendix A). Measuring carrier survival from the transformed text itself is essential—an annotation that merely *asserts* survival reports a spurious perfect score.

10.2 The meaning layer survives real translation (Stage-2)

A closed-domain lexicon extractor over six languages (English, Korean, Hindi, Chinese, Arabic, German) recovers the nine invariant fields from 4,800 real NLLB translations (800 source sentences over 50 documents × six conditions). Aggregate recovery (Wilson 95% CI) is 0.83–0.99: the small closed categoricals (polarity,

quantity, attribution, time) survive at ≥ 0.97, while the open entity and relational fields are lower (0.83–0.92; patient 0.825 weakest). Strict all-nine-field recovery is highest for Hindi/German (0.69/0.66) and lowest for Arabic (0.25), where MT renders entities most variably. Because the recovered fields fix the invariant digest, a meaning-altering edit to any field is detected across languages—direct real-data evidence that the meaning layer survives MT where the surface carriers of Sec. 10.1 do not; a wide-coverage neural parser is the next step (`scripts/eval_multilingual.py`).

10.3 A wide-coverage neural frontend on open-domain text

A feasibility pilot (96 sentences over six conditions) ran an instruction LLM as a drop-in extractor: on *closed* templates the hand-built lexicon leads (0.970 vs. 0.726 field recovery), but on *open-domain* wording the lexicon degrades below the neural frontend (0.481 vs. 0.614), strongest on typed relational fields (predicate 0.896, causation 0.948). The harness enforces *separate* translation and extraction models (removing the self-consistency confound); a translator-independent embedding frontend (`model2vec`) recovers open-domain temporal direction at 0.944 where the lexicon abstains, though a static embedding is insufficient for the full contract.

10.4 Meaning-based provenance authentication under real translation

The producer registers a keyed τ-bit tag (τ=32) over the canonical digest of a declared *invariant contract* (a field subset) in the out-of-band ledger; the verifier re-extracts the invariants (Stage-2) and authenticates iff the tag reproduces. On the 4,800 translations a compact contract (polarity, quantity, time, attribution, predicate, agent) gives document-level attribution—a document attributed when $\geq$ half its sentences authenticate—of 1.000 (CI $[0.929, 1.000]$) for German and Hindi, 0.980 round-trip, 0.940 Korean/Chinese, and 0.900 $[0.786, 0.957]$ Arabic over 50 documents; sentence-level TPR 0.618–0.886, tamper rejection $\geq$ 0.996, false positives 0.000–0.007. Widening to nine fields raises meaning coverage but lowers TPR (AR 0.253) and collapses attribution for the hardest conditions (AR 0.02, ZH 0.08, RT 0.24) while German/Hindi hold at 0.94—an explicit fidelity/robustness trade-off, and the translation-robust counterpart of the 0/192 surface pilot. One caveat is quantitative: because authentication reproduces a tag over a *contract*, two distinct low-entropy generations sharing a contract digest collide before the tag is tested, so the achievable false-positive floor is the *contract-collision* probability set by the contract's entropy, not τ. For the six-field contract this floor is 5.2×10^{-3} ($\approx 1/192$), matching the measured 0.000–0.007 and $\sim 2\times10^{7}\times$ the 2^{-32} bound; nine fields lower it to $\sim 1.8\times10^{-4}$, and the $2^{-\tau}$ bound binds only once the entity-vocabulary entropy exceeds τ bits (Appendix A). The full NLLB run is committed in-repo (`paper/results/realmt_b2/`), so every number reproduces with no GPU (`scripts/eval_provenance.py`).

10.5 Baseline comparison: token, semantic, and robust-hashing

Against a *real* KGW token watermark ((Kirchenbauer et al., 2023); γ=0.25, δ=2.0) generated by a real model and translated by the same NLLB system, detection collapses from 0.960 clean to 0.000–0.380 across six conditions while the meaning-digest survives (0.502–0.980), confirming cross-lingual token-watermark removal (He et al., 2024) (Appendix Table 10). At a *matched* 1% FPR on the same cached real translations (real multilingual embedding), a real semantic baseline (SemStamp (Hou et al., 2024)) survives round-trip (1.000) and a near language (Korean 0.937) but degrades cross-lingually (Hindi 0.617, Chinese 0.460, Arabic 0.377); robust hashing collapses under cross-script translation (exact SHA-256 and token MinHash (Broder, 1997) 0.000; embedding SimHash (Charikar, 2002) 0.000–0.883); the typed meaning-digest survives all six (1.000). Decisively, a single-field meaning-altering edit is rejected by the typed contract (1.000, CI $[0.981, 1.000]$) but by *neither* similarity hash (0.000)—a near-duplicate fingerprint cannot see which typed field changed—while the exact hash that rejects every tamper also rejects every legitimate translation (Table 1).

Table 1: Matched-FPR head-to-head on cached real LLM-MT translations: TPR at a common 1% FPR (300 bootstrap documents of 16 sentences, seeded; real multilingual embedding). RT = English round-trip; KO/HI/ZH/AR = direct translation. SemStamp is an embedding-region watermark; SimHash/MinHash are robust hashes; SHA-256 is an exact hash. Only the typed contract both survives translation and rejects a single-field tamper (1.000 vs. 0.000 for similarity hashes; in text).

Method	RT	KO	HI	ZH	AR
Typed digest	1.000	1.000	1.000	1.000	1.000
SemStamp	1.000	0.937	0.617	0.460	0.377
SimHash	1.000	0.590	0.000	0.883	0.633
MinHash	1.000	0.000	0.000	0.000	0.000
SHA-256	0.867	0.000	0.000	0.000	0.000

10.6 Ledger retrieval and NLI baselines

Because the scheme already uses an out-of-band ledger, the natural control is to store the source and *retrieve* it after transformation (nearest neighbor over a sentence embedding), optionally with an NLI check. Under the same ledger and id-addressed verification with benign acceptance matched at ≈ 0.95, plain retrieval and a directional-coverage NLI proxy reject essentially no single-field tamper (0.047/0.061): a one-field edit stays nearest its own source. Only a *perfect*, unrealizable oracle NLI matches the typed contract's 1.000 rejection, and even then neither localizes the changed field nor yields a keyed tag, both of which the typed digest provides with no model (Appendix Table 11).

10.7 Paraphrase and adaptive-attack robustness

Benign meaning-preserving paraphrase authenticates at TPR 1.000 and document attribution 1.000 (RQ4), and a single-field altering edit is rejected at 1.000. A schema-aware adaptive attacker (RQ7) that swaps the predicate verb and time expression for *out-of-lexicon* synonyms achieves ValidRemoval (Eq. (1)) 1.000 under the *closed* lexicon—an honest negative we localize to exactly the abstained fields (predicate and time in all cases; attribution when present), with polarity and agent recovered in 100% of attacks. The vulnerability is the fixed lexicon, not the meaning-digest: an *extended-coverage* frontend drives it $1.000 \rightarrow 0.000$ at unchanged benign and tamper rejection, but a *held-out* out-of-inventory attack still evades any fixed lexicon (1.000). The general defense is open-vocabulary parsing: two real instruction LLMs (schema-guided `neural_parser`) drive the held-out novel ValidRemoval to 0.050 (GPT-4o-mini, N=40) and 0.100 (Llama-3.3-70B, N=10), both disjoint from the lexicons' upper CIs, at benign and tamper rejection 1.000 (Appendix Tables 9, 12).

11 Conclusion

Truthprint is a language-neutral, typed semantic IR for authenticated provenance whose key contribution is a machine-checkable separation between immutable meaning and controlled realization choices. What survives transformation is ledger-based authenticated semantic provenance—a keyed tag over the invariant contract, re-verified after translation or paraphrase—complementary to, not a replacement for, token-level watermarks. It cannot guarantee detection after arbitrary rewriting, but targets a narrower, defensible objective—preserving provenance across transformations that retain substantial meaning—which the real-MT evidence supports; wide-coverage parsing, official baselines, and scale remain future work.

Limitations

Multilingual semantic parsing remains imperfect. Not every proposition admits multiple safe realization alternatives. Highly constrained text has low carrier capacity. Some apparently stylistic alternations change emphasis or information structure. Translation can omit modality, evidentiality, subjects, or discourse relations. Strong summarization can destroy both content and watermark evidence. Finally, candidate generation and semantic validation add computational overhead compared with direct token-level methods. The closed-domain adaptive paraphrase of Section 10.7 shows the sharpest limitation directly: a schema-aware attacker who swaps out-of-lexicon content synonyms fully removes the meaning-digest under the closed lexicon while preserving meaning. An extended-coverage frontend defends that specific attack (ValidRemoval $1.000 \rightarrow 0.000$ without weakening tamper rejection); a held-

out out-of-inventory attack still evades any fixed lexicon, but two real open-vocabulary neural runs (GPT-4o-mini N=40, Llama-3.3-70B N=10; Table 9) defend even the held-out attack (0.050/0.100), establishing that wide-coverage *open-vocabulary* parsing—not a longer lexicon—is the general defense. Those runs are closed-domain and modest in scale; the current artifact still does not evaluate real generators, human judgments, all official baselines, replay defenses, or open-vocabulary adaptive attacks *at larger scale and across domains*; consequently it supports feasibility and closed-domain robustness claims, not comparative robustness or deployment readiness at wide coverage.

Ethical Considerations

A provenance detector can be misused to accuse authors, censor content, or make automated disciplinary decisions. Truthprint therefore treats a positive result as evidence about a participating keyed generation system, not proof of AI authorship or of misconduct. Deployments should expose the tested scheme and key identifier, use independently calibrated error rates, preserve an appeal path, and avoid adverse decisions based on the detector alone. Absence of a mark is likewise not evidence of human authorship.

The design has dual-use risks: public carrier families can aid removal attacks, while secret or overly detailed payloads can enable tracking. Payloads should exclude personal identifiers, prompts, and precise timestamps; keys and nonce registries require access control, rotation, and audit. Evaluation text should be licensed or consented, human annotation should be compensated and documented, and language-specific error disparities should be reported rather than hidden in an aggregate score. The present artifact uses synthetic template facts, no personal data, and no human subjects; the real-MT pilot (Secs. 10.1–10.4) translates those synthetic facts with a public open-weights model (NLLB-200-distilled-600M) and involves no personal data or human subjects. Any scaled evaluation that adds human invariant or factual-equivalence annotation must use licensed or consented text and compensated, documented annotation.

References

Andrei Z. Broder. 1997. On the resemblance and containment of documents. In *Proc. Compression and Complexity of Sequences*, pages 21–29.

Moses S. Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *Proc. ACM STOC*, pages 380–388.

Yize Cheng, Vinu Sankar Sadasivan, Mehrdad Saberi, Shoumik Saha, and Soheil Feizi. 2025. Adversarial paraphrasing: A universal attack for humanizing AI-generated text. In *Proc. NeurIPS*.

Miranda Christ, Sam Gunn, and Or Zamir. 2024. Undetectable watermarks for language models. In *Proc. COLT*.

Sumanth Dathathri, Abigail See, Sumedh Ghaisas, Po-Sen Huang, Rob McAdam, Johannes Welbl, Vandana Bachani, Alex Kaskasoli, Robert Stanforth, Tatiana Matejovicova, Jamie Hayes, Nidhi Vyas, Majd Al Merey, Jonah Brown-Cohen, Rudy Bunel, Borja Balle, Taylan Cemgil, Zahra Ahmed, Kitty Stacpoole, and 5 others. 2024. Scalable watermarking for identifying large language model outputs. *Nature*, 634(8035):818–823.

European Parliament and Council. 2024. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union.

Pierre Fernandez, Antoine Chaffin, Karim Tit, Vivien Chappelier, and Teddy Furon. 2023. Three bricks to consolidate watermarks for large language models. In *Proc. IEEE WIFS*.

Thibaud Gloaguen, Nikola Jovanović, Robin Staab, and Martin Vechev. 2025. Black-box detection of language model watermarks. In *Proc. ICLR*.

Google. 2026. SynthID: Tools for watermarking and detecting LLM-generated text. Responsible Generative AI Toolkit, Google AI for Developers.

Google DeepMind. 2024. SynthID-Text reference implementation. GitHub repository.

Xia Han, Qi Li, Jianbing Ni, and Mohammad Zulkernine. 2025. Robustness assessment and enhancement of text watermarking for Google's SynthID. In *Proc. IEEE TrustCom*.

Zhiwei He, Binglin Zhou, Hongkun Hao, Aiwei Liu, Xing Wang, Zhaopeng Tu, Zhuosheng Zhang, and Rui Wang. 2024. Can watermarks survive translation? on the cross-lingual consistency of text watermark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4115–4129.

Abe Bohan Hou, Jingyu Zhang, Tianxing He, Yichen Wang, Yung-Sung Chuang, Hongwei Wang, Lingfeng Shen, Benjamin Van Durme, Daniel Khashabi, and Yulia Tsvetkov. 2024. SemStamp: A semantic watermark with paraphrastic robustness for text generation. In *Proc. NAACL*.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. In *Proc. ICML*.

Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. 2024. Robust distortion-free watermarks for language models. *Transactions on Machine Learning Research*.

Aiwei Liu, Leyi Pan, Xuming Hu, Shu'ang Li, Lijie Wen, Irwin King, and Philip S. Yu. 2024a. An unforgeable publicly verifiable watermark for large language models. In *Proc. ICLR*.

Aiwei Liu, Leyi Pan, Xuming Hu, Shiao Meng, and Lijie Wen. 2024b. A semantic invariant robust watermark for large language models. In *Proc. ICLR*.

Aiwei Liu, Leyi Pan, Yijian Lu, Jingjing Li, Xuming Hu, Xi Zhang, Lijie Wen, Irwin King, Hui Xiong, and Philip S. Yu. 2024c. A survey of text watermarking in the era of large language models. *ACM Computing Surveys*, 57(2):1–36.

Romina Omidi, Yun Dong, and Binghui Wang. 2026. On Google's SynthID-Text LLM watermarking system: Theoretical analysis and empirical validation. *Preprint*, arXiv:2603.03410.

Leyi Pan, Aiwei Liu, Zhiwei He, Zitian Gao, Xuandong Zhao, Yijian Lu, Binglin Zhou, Shuliang Liu, Xuming Hu, Lijie Wen, Irwin King, and Philip S. Yu. 2024. MarkLLM: An open-source toolkit for LLM watermarking. In *Proc. EMNLP (System Demonstrations)*.

Wenjie Qu, Wengrui Zheng, Tianyang Tao, Dong Yin, Yanze Jiang, Zhihua Tian, Wei Zou, Jinyuan Jia, and Jiaheng Zhang. 2025. Provably robust multi-bit watermarking for AI-generated text. In *Proc. USENIX Security*.

Jie Ren, Han Xu, Yiding Liu, Yingqian Cui, Shuaiqiang Wang, Dawei Yin, and Jiliang Tang. 2024. A robust semantics-based watermark for large language model against paraphrasing. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 613–625.

Tom Sander, Pierre Fernandez, Alain Durmus, Matthijs Douze, and Teddy Furon. 2024. Watermarking makes language models radioactive. In *Proc. NeurIPS*.

Jonas Schäfer, Cezary Pilaszewicz, and Gerhard Wunder. 2026. Robust text watermarking for large language models via dual semantic embeddings. In *Findings of the Association for Computational Linguistics: EMNLP 2026*.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. 2024. AI models collapse when trained on recursively generated data. *Nature*, 631(8022):755–759.

Ziping Ye, Gourab Dey, Christos Christodoulopoulos, Charith Peris, Anil Ramakrishna, Weitong Ruan, Aram Galstyan, Kai-Wei Chang, Rahul Gupta, and Ninareh Mehrabi. 2026. SWAN: Semantic watermarking with Abstract Meaning Representation. In *Proc. ACL*.

Xin Yi, Yue Li, Shunfan Zheng, Linlin Wang, Xiaoling Wang, and Liang He. 2025. Unified attacks to large language model watermarks: Spoofing and scrubbing in unauthorized knowledge distillation. *Preprint*, arXiv:2504.17480.

Xuandong Zhao, Prabhanjan Ananth, Lei Li, and Yu-Xiang Wang. 2024. Provable robust watermarking for AI-generated text. In *Proc. ICLR*.

A Reference Implementation and Verification

To make the guarantees above concrete and reproducible, we provide a self-contained reference implementation (`reference/truthprint_poc.py`, standard library only) of the language-independent core: the canonical invariant digest h_I, the authenticated payload $p = m \parallel \mathrm{MAC}_K(\cdot)$, the invariant-bound keyed carrier map, and a systematic GF(2) code with erasure decoding. This deliberately excludes the semantic parser so that the cryptographic and coding properties can be validated in isolation from model-dependent NLP. It exercises four properties and reports the following verified behavior (message $\mu = 16$ bits, tag $\tau = 32$ bits, code $[N, k] = [96, 48]$):

- **P1 (round-trip recovery).** An invariant-preserving transform that re-realizes carriers and erases 28/96 sites still yields exact payload recovery and a verifying MAC.

- **P2 (invariant binding).** Flipping a locked field (polarity positive→negative) changes h_I, so the MAC fails and the document is *not* attributed, illustrating Proposition 1's contract.

Table 2: Reference-implementation decode success vs. carrier erasure rate ($[96, 48]$ GF(2) code, 500 trials per point, seeded). Fine resolution near the cliff reveals a gradual transition from full recovery at $\epsilon = 0.42$ to collapse at $\epsilon \geq 0.52$, as predicted by Proposition 2.

Carrier erasure ϵ	Recovery rate
0.10	1.000
0.20	1.000
0.30	1.000
0.40	1.000
0.42	0.998
0.44	0.982
0.45	0.968
0.46	0.950
0.48	0.774
0.50	0.290
0.52	0.000
0.55	0.000
0.60	0.000

- **P3 (cryptographic false positives).** Over 20,000 independent documents, 0 were attributed. This is consistent with Theorem 2; statistically, the observation alone only gives an approximate 95% binomial upper bound of 1.5×10^{-4}.

- **P4 (no global rule).** For a fixed symbol, $43/96 \approx N/2$ realized options differ between two documents whose invariants differ, confirming the map is bound to (K, h_I, n).

Table 2 reports decode success versus carrier erasure rate for the implemented $[96, 48]$ code and seeded random-erasure protocol: full recovery (≥ 0.998) through $\epsilon = 0.42$, followed by 0.982 at $\epsilon = 0.44$, 0.774 at $\epsilon = 0.48$, 0.290 at $\epsilon = 0.50$, and zero observed recovery at $\epsilon \geq 0.52$. The location and shape are finite-code measurements; only the necessary survivor-count boundary follows from Proposition 2. These values characterize the coding layer, not detection on model-generated text.

A.1 Closed-Domain Linguistic Round-Trip

The core PoC operates on abstract carrier bits. To validate the linguistic layer end to end, a second self-contained script (`reference/truthprint_linguistic_demo.py`) realizes an authenticated payload across templated factual sentences using two invariant-preserving carriers—voice (active/passive) and time-phrase position—then rewrites, re-parses, and detects on actual sentence strings (the running example of Section E). On a 32-sentence document (64 carriers, code $[64, 32]$, $\tau = 20$) it verifies:

- **L1 (invariant fidelity).** Every benign transform leaves the parsed agent, predicate, patient, and polarity unchanged.

- **L2 (round-trip recovery).** With 19/64 carriers erased by realization-canonicalizing rewrites, the exact payload is recovered and the MAC verifies.

- **L3 (tamper detection).** Negating a single fact changes the document invariant digest, so attribution correctly fails.

This is a Stage-1 closed-domain demonstration, not a general multilingual parser; it establishes that the contract, keyed carriers, and erasure-aware decoder compose correctly once realized as text, and it isolates the remaining open problem as robust wide-coverage semantic parsing (Section B).

A.2 Field-Level Challenge Set and Component Ablation

To test—rather than assume—that a *typed* invariant contract catches what an embedding *similarity* judge misses, we extend the closed-domain frontend to six independently protected fields (polarity, quantity, temporal direction, modality, attribution, causation) and generate minimal-pair probes (`reference` is packaged as `truthprint.challenge`; regenerate with `scripts/eval_challenge.py`). For each field we compare a meaning-preserving realization change against a single-field meaning-altering edit, and judge each with three detectors that differ only in the admissibility rule: the full typed+MAC contract, a decode-only ablation that drops the h_I-bound MAC, and an embedding-similarity gate (bag-of-words cosine) *calibrated* to reject at most 5% of legitimate paraphrases. On 300 documents of 24 facts ($[48, 24]$ code), with 95% confidence intervals (Wilson for rates, bootstrap for cosines) at a fixed seed, we measure:

- **C1 (typed tamper detection).** The typed+MAC contract detects every single-field tamper across all six fields (aggregate 1.000, CI $[0.998, 1.000]$) while retaining attribution across benign paraphrases (0.990, CI $[0.971, 0.997]$).

- **C2 (MAC is load-bearing).** The decode-only ablation detects *no* tamper (aggregate 0.000, CI $[0.000, 0.002]$): the carriers still recover, so the h_I-bound MAC—not the coding layer—is what resists a meaning-altering edit.

- **C3 (similarity cannot separate meaning).** The calibrated embedding gate catches almost no tamper (aggregate 0.012, CI $[0.008, 0.018]$), because a single-field edit is lexically smaller than a legitimate voice/time reflow: in 48.6% of cases (CI $[0.463, 0.509]$) the tamper is at least as cosine-similar to the source as the paraphrase is, so *no* similarity threshold can separate them.

This isolates the interaction the design relies on—typed invariants supply the decision an embedding metric cannot, and the MAC binding converts that decision into tamper resistance—on real strings. It remains a closed-domain result; the wide-coverage multilingual analogue requires the neural frontend and human annotations specified in Section B and the accompanying data templates.

B Experimental Methodology

B.1 Research Questions

We propose the following research questions:

- RQ1: Does Truthprint preserve immutable meaning better than unconstrained paraphrase-based watermarking?

- RQ2: What true-positive rate is achieved at fixed false-positive rates on clean text?

- RQ3: How much signal survives single-hop and round-trip translation?

- RQ4: How robust is Truthprint to sentence- and paragraph-level paraphrasing?

- RQ5: What quality and latency costs are introduced?

Table 3: Evaluation matrix: each research question mapped to its decisive comparison, manipulated factor, and primary outcome measure.

RQ	Decisive comparison	Primary measure
RQ1	Truthprint vs. unconstrained paraphrase watermark	Invariant error rate (IER), contradiction rate
RQ2	Truthprint vs. token/embedding baselines, clean text	TPR at $\{1, 0.1, 0.01\}$% FPR; ROC-AUC
RQ3	Pre- vs. post-translation (1-hop, round-trip)	Payload BER, partial-recovery rate
RQ4	Sentence- vs. paragraph-level paraphrase	TPR@1%FPR vs. edit distance
RQ5	Watermarked vs. unwatermarked generation	Quality delta, added latency
RQ6	Recovery vs. document length	Authenticated bits recovered
RQ7	Adaptive attacker with vs. without invariant constraint	ValidRemoval rate (Eq. (1))

- RQ6: How many authenticated payload bits can be recovered as document length increases?

- RQ7: Can an adaptive attacker remove the watermark while preserving all locked invariants?

Table 3 fixes, for each question, the single manipulated factor and the outcome that would confirm or falsify the corresponding hypothesis, so that results cannot be reported selectively. RQ4 (paraphrase) and RQ7 (adaptive removal, Eq. 1) are measured on the closed domain in Section 10.7; the remaining questions await the wide-coverage frontend.

B.2 Diagnostic Simulation (Not an End-to-End Benchmark)

A controlled Stage-1 simulator checks the evaluation code paths and illustrates, under explicitly stipulated channel parameters, how signal placement (token vs. meaning layer) reacts to surface replacement and carrier erasure. Because the "baselines" are method-inspired proxies run from a shared harness rather than the published systems, the simulator *must not be used to rank methods* and is not evidence of real-translation performance. To keep this diagnostic from being mistaken for a benchmark,

its full protocol and outputs (Tables 4 and 5) are deferred to Appendix F; the measured results this paper relies on are the real-MT findings of Sections 10.1–10.4 and the coding-core verification of Section A.

C Falsifiable Hypotheses

The following are hypotheses to be tested at scale with the neural frontend. All hypotheses remain unconfirmed until evaluated on wide-coverage model text with real transformation systems:

- H1: Truthprint will outperform token-level watermarking under translation because the carrier is reconstructed from language-independent semantics.
- H2: Explicit invariant validation will reduce errors in negation, quantity, time, causality, and attribution.
- H3: Truthprint may underperform token methods on very short text because semantic carrier capacity accumulates across sentences.
- H4: ECC and erasure-aware decoding will improve recovery under parsing errors and partial deletion.
- H5: Strong summarization will remain difficult because it removes propositions and their associated carriers.
- H6: Combining structured carriers with embedding-based fallback evidence will outperform either alone across diverse languages.

D Security and Deployment Considerations

Carrier mappings should depend on the secret key, invariant digest, document nonce, and carrier identity. This prevents simple global carrier rules and reduces forgery risk. Exact detector scores should be access controlled or coarsened to reduce oracle attacks. The detector should localize evidence by paragraph and avoid attributing a mixed document to one source.

The reference API supplies the nonce to the detector out of band; natural-language carrier recovery alone does not discover it. A deployment must therefore define an authenticated nonce-resolution mechanism, such as a bounded time-bucket search plus deployment identifier or a provenance-ledger lookup. Every tested key–nonce pair consumes false-positive budget and must be included in calibration. A ledger should reject duplicate nonces and distinguish exact replay from fresh attribution. Splicing requires finer-grained binding (for example, paragraph identifiers and a document Merkle root); without it, a valid paragraph proves only that the participating generator produced that paragraph, not the provenance of the surrounding document.

Watermarks should support transparency rather than automated punishment. Every detection report should include the tested key and scheme version, text length, number of reliable carriers, calibrated false-positive rate, language and attack assumptions, confidence, and MAC verification status. User identifiers, raw prompts, and exact timestamps should not be embedded.

D.1 Deployment Path and Graceful Degradation

Truthprint is designed to slot into a serving stack as a semantic realization pass rather than as a modification of the token sampler. Two integration modes are practical. In the *inline* mode, the generator emits a draft, the encoder re-realizes admissible discourse units to carry payload symbols, and the validator gates each candidate; although the coding and cryptographic layers are independent of model size (Sec. 9), the unmeasured generation and validation overhead may be substantial. In the *asynchronous* mode, watermarking runs as a post-generation service, which decouples watermark latency from the user-facing generation latency at the cost of a short delay before provenance is finalized.

Because capacity is unevenly distributed across text (Eq. 5), a deployable system must degrade gracefully. Low-capacity spans—code, formulas, legal clauses, and verbatim quotations—admit few invariant-preserving realizations and are therefore poor carriers. For such spans Truthprint records erasures and can optionally fall back to a token-level watermark such as SynthID-Text, yielding a layered scheme intended to combine semantic-

layer transformation tolerance with token-layer short-span coverage; both properties require end-to-end evaluation. The detector localizes evidence per paragraph, reports the number of reliable carriers alongside the calibrated false-positive rate, and abstains rather than guessing when the recovered capacity falls below the minimum reliable length. This makes the system's competence explicit at detection time, which is the behavior an auditor or platform integrator requires.

E Worked Example

Consider the fact with locked invariants FIX(agent=developer, patient=server_error), polarity positive, time *previous_day*. Two binary carriers are available: VOICE $\in$ {active, passive} and TIME-POS $\in$ {front, end}. Suppose the keyed map assigns the code symbols so that this unit must realize as passive with the time phrase at the front:

> *"On the previous day, the server error was fixed by the developer."*

A benign transformation—say an English→Korean translation—typically preserves the invariants but may not preserve every carrier. If translation canonicalizes voice (Korean commonly renders this actively), the VOICE carrier is destroyed; the detector, on re-parsing, flags it as *low confidence* and records an *erasure* rather than reading a wrong bit. The TIME-POS carrier and the carriers of neighboring sentences survive. Because the payload is spread across sentences and protected by an error-correcting code, the authenticated payload is still recovered once the surviving carriers exceed the capacity floor of Eq. (5). In contrast, a rewrite that inserts a negation ("was *not* fixed") alters a locked field, changes the invariant digest h_I, and correctly fails MAC verification. This example is executed end-to-end on sentence strings in the reference implementation (Section A, properties L1–L3).

F Diagnostic Simulation: Full Protocol and Outputs

This appendix gives the full protocol and outputs of the controlled Stage-1 diagnostic summarized in the main text. It validates the simulator implementation only; it is not a neural or translation benchmark and must not be used to rank the named published systems.

Tables 4 and 5 report simulator outputs for Truthprint *and* four baselines on a shared closed-domain Stage-1 testbed (500 documents, 32 sentences each). To keep the comparison apples-to-apples in the absence of a wide-coverage neural frontend, every method is evaluated under one identical meaning-preserving channel and one identical protocol; the baselines are method-inspired proxies for selected aspects of each published method's signal placement, run from the same harness (`scripts/eval_baselines.py`), and abstract the neural frontend (LM sampler, sentence encoder, AMR parser) in exactly the way the Truthprint Stage-1 numbers do. This diagnostic checks code paths and illustrates hypotheses under explicitly chosen channel parameters. It is not evidence that any named method attains the reported performance on real translation, and it must not be used to rank the methods. End-to-end evaluation with real generators, translation systems, parsers, official baseline implementations, held-out calibration data, and confidence intervals remains future work.

Shared channel. A single meaning-preserving transform is characterised by two physical quantities that all methods see identically: the fraction τ_{tok} of surface tokens the transform replaces (cross-script translation replaces almost all tokens) and the fraction ϵ_{inv} of meaning-layer carriers lost to parse normalisation (the quantity Truthprint's own analysis uses). Each method's survival is then *derived* from these two numbers through its own signal dependence, so the relative ordering is forced by where the signal lives, not tuned per method: a KGW-style token-identity green-list proxy survives at $(1-\tau_{\text{tok}})^2$; an edit-aligned token watermark (DEW) at $(1-\tau_{\text{tok}})$ up to an edit budget then collapses; a sentence-embedding region (SemStamp) flips near the LSH boundary at a rate $\propto \tau_{\text{tok}}$ but is meaning-dominant; and meaning-layer signals (SWAN, Truthprint) are unaffected by τ_{tok}, losing only the ϵ_{inv} parse fraction.

Protocol. For each method a decision threshold is calibrated at 1% FPR on null (unwatermarked) documents; TPR is measured at

Table 4: Diagnostic clean-channel simulation (500 docs × 32 sentences; threshold at 1% FPR)†. Detection saturates for all methods on clean text; the discriminating transformed setting appears in Table 5.

Method	TPR@1%FPR	ROC-AUC	BLEU-1
KGW proxy	1.000	1.000	—
SemStamp	1.000	1.000	—
SWAN	1.000	1.000	—
Truthprint	1.000	1.000	0.84

†Controlled simulation only, not a neural or translation benchmark. BLEU-1 measured only for Truthprint (real sentences); baseline fluency needs each method's neural generator. Truthprint additionally applies a 20-bit tag check; this is not a measured end-to-end FPR.

Table 5: Diagnostic transformed-channel simulation (TPR at 1% FPR) $(\tau_{\text{tok}}, \epsilon_{\text{inv}})$ per language pair†: EN→KO $(0.92, 0.30)$, EN→HI $(0.95, 0.35)$, KO→EN $(0.90, 0.28)$. Outputs illustrate the stipulated channel equations, not real translation.

Method	EN→KO	EN→HI	KO→EN
KGW proxy	0.020	0.018	0.030
DEW	0.000	0.000	0.000
SemStamp	0.996	0.988	0.994
SWAN	1.000	1.000	1.000
Truthprint	1.000	0.972	0.998

†Controlled Stage-1 channel; τ_{tok} (token replacement) and ϵ_{inv} (meaning-carrier erasure) per translation event. Only the Truthprint proxy in this harness exercises authenticated recovery; this table does not evaluate baseline security.

that threshold after the channel; ROC-AUC is computed from watermarked-vs-null score populations. Truthprint is run through its real linguistic codec on actual sentences ($\mu = 12$, $\tau = 20$, $[64, 32]$ GF(2) code); its quality is unigram BLEU between the watermarked and an unconstrained realization of the same facts. Baseline fluency/quality is not measured because it requires each method's neural generator, absent from the Stage-1 abstraction.

Reading the tables. The clean setting saturates because the proxy generators deliberately select matching symbols and provide many carriers. Under the stipulated high-replacement setting, token proxies lose signal by construction while semantic proxies retain it according to the chosen erasure parameter. These outcomes validate the simulator implementation, not H1 and not any comparison among the named published systems. P2 and P3 separately exercise Truthprint's authenticated coding core (Section A); this harness does not evaluate the security properties of the other systems.

G Reproducibility and Notation

This section pins every choice needed to reimplement the language-independent core exactly, so that the reference scripts and the paper agree symbol for symbol. Nothing here is left to interpretation; the two scripts in `reference/` realize precisely these definitions.

G.1 Primitive Instantiation

The reference implementation fixes the following concrete choices.

- **Invariant digest.** $h_I = \text{SHA-256}(\text{JSON}(I))$, where JSON is canonical: keys sorted, separators "`,`" and "`:`", UTF-8 encoded. Canonicalization makes h_I order- and whitespace-independent.

- **Authentication.** MAC = HMAC-SHA256; the tag is the first τ bits of the MAC over $m \parallel n \parallel h_I$. The payload is $p = m \parallel \text{tag}$ (length $|p| = \mu + \tau$).

- **Keyed carrier map (binary carriers).** $\pi_j = \text{LSB}\big(\text{HMAC-SHA256}(K, h_I \parallel n \parallel \text{ID}_j)\big)$ with ID_j the 4-byte big-endian carrier index. The realized option is $a_j^\star = \pi_j \oplus c_j$ and the recovered symbol is $\hat{c}_j = \pi_j \oplus a_j^{obs}$.

- **Error-correcting code.** A systematic $[N, k]$ linear code over GF(2) with generator $G = [\, I_k \mid P \,]$, where P is drawn once from a fixed pseudo-random seed. Erasure decoding solves the surviving linear equations by Gaussian elimination over GF(2); it succeeds iff the retained rows have rank k (Proposition 2).

- **Detection decision.** Attribute iff erasure decoding returns some $\hat{p}$ *and* MAC verifies on $(\hat{m}, n, h_I)$; otherwise report the calibrated statistic z of Eq. (2) against a threshold τ_z without asserting authenticated attribution.

G.2 Notation-to-Code Map

Table 6 maps paper symbols to identifiers in `reference/truthprint_poc.py`, removing any ambiguity between the formalism and the code.

Table 6: Mapping between paper notation and reference-implementation identifiers.

Symbol	Meaning	Code identifier
h_I	canonical invariant digest	`canonical_invariant_digest`
p	authenticated payload	`authenticate_payload`
MAC	tag verification	`verify_payload`
ECC	systematic GF(2) code	`GF2Code.encode`
decode	erasure decoding	`GF2Code.decode_erasure`
π_j	keyed carrier offset	`keyed_bit`
$a_j^\star$	realized option	`realize_option`
$\hat{c}_j$	recovered symbol	`recover_symbol`
D_K	detector	`detect_document`

G.3 Parameters, Determinism, and How to Run

The core script uses $\mu = 16$, $\tau = 32$, $[N, k] = [96, 48]$; the linguistic script uses $\mu = 12$, $\tau = 20$, $[N, k] = [64, 32]$ over 32 sentences with two carriers each. All randomness is seeded: message bits and erasure sampling use fixed seeds, and the code matrix P uses a fixed seed, so every reported number is reproducible. The document nonce n is freshly sampled per run and passed explicitly to the detector; recovery is invariant to its value, so success does not depend on the nonce. Both scripts return a nonzero exit code if any asserted property fails, so they double as regression tests. Reproduction requires only Python 3 (standard library only); the paper builds with `pdflatex` via `make`. The exact commands are:

```
python3 reference/truthprint_poc.py
        # P1-P4
python3 reference/truthprint_linguistic_demo.
    py # L1-L3
make
        # build main.pdf
```

Expected results are those reported in Section A: P1–P4 hold, and L1–L3 hold on sentence strings.

The same primitives are also packaged as an installable Python distribution (`code/` in the accompanying repository) with a unit-test suite and continuous integration. Its command-line interface reruns all properties (`truthprint selftest`) and regenerates Table 2 deterministically from a fixed seed with 500 trials per erasure rate (`truthprint repro-table`).

H Computational Complexity

Let a document have S discourse units, $|C|$ carriers, κ candidates per carrier, and N code positions. Encoding cost is $O(S \cdot |C| \cdot \kappa \cdot c_{\text{val}})$, dominated by the validator cost c_{val} (a parse plus entailment check). Detection cost is $O(S \cdot c_{\text{parse}} + N \cdot c_{\text{map}} + D_{\text{ECC}})$, where the keyed map c_{map} is one HMAC evaluation and $D_{\text{ECC}} = O(Nk)$ for GF(2) erasure elimination (worst case $O(Nk^2)$). The cryptographic and coding operations scale with the number of carriers and are independent of the generator parameter count. End-to-end latency is nevertheless unknown because candidate generation, reparsing, and validation dominate and may themselves invoke large models; it must be measured for each implementation.

I Encoding and Detection Pseudocode

Listing 1: Truthprint encoding pseudocode.

```
G <- BuildTruthprint(x)
I <- ExtractInvariants(G)
p <- AuthenticatePayload(K, m, nonce, Hash(I)
    )
c <- ErrorCorrectingEncode(p)
C <- DiscoverAdmissibleCarriers(G)
for assignment in KeyedAssign(K, I, C, c):
    G2 <- ApplyCarrier(G, assignment)
    candidates <- RealizeCandidates(G2,
    language)
    accepted <- ValidateAgainstContract(
    candidates, I)
    if accepted is empty:
        MarkErasure(assignment)
    else:
        CommitBestCandidate(accepted)
return AssembleDocument()
```

Listing 2: Truthprint detection pseudocode.

```
segments <- SegmentAndIdentifyLanguage(text)
observations <- []
for segment in segments:
    G, conf <- ParseTruthprint(segment)
    I <- CanonicalizeInvariants(G)
    for carrier in ExtractObservableCarriers
    (G):
        if Reliability(carrier, conf) >=
    threshold:
            observations.append(
    InvertKeyedMap(K, I, carrier))
        else:
            observations.append(ERASURE)
candidates <- ECCDecode(observations)
valid <- VerifyMACs(K, candidates)
score <- CalibratedStatisticalScore(
    observations, K)
return score, Best(valid), ConfidenceReport()
```

J Practical Software Architecture

A prototype can use the following modular layout:

```
truthprint/
  schema/        # entities, events,
    relations, mutability
  parser/        # multilingual parser and
    canonicalizer
  carriers/      # voice, packaging,
    discourse, ordering
  encoding/      # payload, keyed mapping,
    ECC
  realization/   # candidate generation and
    ranking
  validation/    # hard constraints, NLI,
    numbers, quality
  detection/     # extraction, decoding,
    statistics
  attacks/       # translation, paraphrase,
    summary, splice
  experiments/   # datasets, metrics,
    runners
```

J.1 Core Data Model

Listing 3: Illustrative Python data model.

```
from enum import Enum
from pydantic import BaseModel, Field

class Mutability(str, Enum):
    LOCKED = "locked"
    CONDITIONAL = "conditional"
    REALIZATION_ONLY = "realization_only"

class Event(BaseModel):
    id: str
    predicate: str
    roles: dict[str, str]
    polarity: str = "positive"
    modality: str = "asserted"
    time: dict = Field(default_factory=dict)
    quantity: dict | None = None
    attribution: str | None = None

class CarrierSite(BaseModel):
    carrier_id: str
    family: str
    target_node_ids: list[str]
    alternatives: list[str]
    recoverability: float
    quality_cost: float
```

J.2 Keyed Mapping Skeleton

Listing 4: Secret-keyed carrier assignment skeleton.

```
import hmac
from hashlib import sha256

def keyed_index(key, invariant_hash, nonce,
                carrier_id, symbol,
    option_count):
    msg = b"|".join([
        invariant_hash,
        nonce,
        carrier_id.encode(),
        symbol.to_bytes(4, "big")
    ])
    digest = hmac.new(key, msg, sha256).
    digest()
    return int.from_bytes(digest[:8], "big")
     % option_count
```

K Prototype Roadmap

K.1 Stage 1: Closed-Domain Proof of Concept

Begin with technical or news-style factual statements. Support entities, events, agent/patient roles, polarity, number, time, and causality. Start with carriers that are easy to validate: active/passive voice, proposition order for independent facts, causal connective family, clause splitting/merging, and nominal/verbal realization.

K.2 Stage 2: Multilingual Parsing

Map English, Korean, and Hindi into one predicate and role inventory using a combination of multilingual structured extraction, dependency parsing, semantic-role labeling, named-entity normalization, numerical and temporal normalization, and multilingual embeddings as a fallback alignment mechanism.

K.3 Stage 3: Document-Level Coding

Introduce multi-ary carrier alphabets, BCH or Reed–Solomon codes, carrier interleaving, synchronization markers, order-independent anchors, partial payload recovery, and provenance database lookup.

L Extended Evaluation Protocol

The following enumerations complete the falsifiable protocol summarized in Section B; they are the baselines, languages, attacks, and metrics a full evaluation must cover.

L.1 Baselines

Experiments should include unwatermarked decoding, a representative Red–Green token watermark, SynthID-Text, SemStamp, SemaMark, semantic-invariant watermarking, SWAN, DEW, Truthprint without invariant validation, Truthprint without ECC, embedding-only Truthprint, and structure-only Truthprint. Standardized baseline implementations from an open watermarking toolkit should

be used where available to ensure comparability (Pan et al., 2024).

L.2 Languages and Domains

Recommended languages include English, Korean, Hindi, Chinese, Arabic, and German or French. Domains should include factual news, encyclopedic explanations, technical documentation, scientific summaries, policy-style prose, dialogue, and creative writing. Code, formulas, legal clauses, and direct quotations should be reported separately due to low semantic carrier capacity.

L.3 Attacks

The evaluation should cover Unicode and punctuation edits, spelling correction, direct translation, round-trip and two-hop translation, lexical and syntactic paraphrasing, sentence splitting/merging, proposition reordering, deletion, insertion, splicing, summarization at multiple compression ratios, regeneration from bullet points, repeated detector-guided paraphrasing, and carrier-targeted rewriting.

L.4 Metrics

Detection metrics include ROC-AUC, true-positive rate at 1%, 0.1%, and 0.01% false-positive rates, false-negative rate, payload bit-error rate, complete and partial recovery rates, minimum reliable text length, and detector latency.

Semantic integrity metrics include entity consistency, role consistency, polarity accuracy, numerical accuracy, temporal consistency, causal-direction consistency, attribution consistency, bidirectional entailment, contradiction rate, and human factual-equivalence judgment.

A weighted invariant error rate can be defined as

$$\mathrm{IER} = \frac{\sum_i w_i E_i}{\sum_i w_i}, \tag{6}$$

where critical categories such as polarity and numeric value receive larger weights or are treated as hard failures.

Table 7: Claim–evidence map for the present submission. "Not established" means that the claim must not be inferred from the diagnostic simulation.

Claim	Status	Supporting evidence or missing test
Coding/ authentication round trip	Demonstrated	Deterministic core tests P1–P4 and closed-domain string tests L1–L3.
Protected-field fidelity	Conditional (validator soundness measured)	Proposition 1, assuming a sound invariant extractor and validator. The soundness assumption is measured as the validator's false-accept rate on meaning-altering edits: 0 observed on the closed domain (C1) and ≤ 0.01 under real MT (Sec. 10.4); wide-coverage soundness remains open.
Fresh-payload unforgeability	Conditional	Reduction to EUF-CMA security of the MAC; replay and nonce discovery are separate deployment concerns.
Surface-carrier translation robustness	Refuted (real MT)	Stage-1 surface parser recovers 0/192 real NLLB translations (Sec. 10.1).
Meaning-layer recovery under real MT	Demonstrated (closed-domain, six languages; data committed)	Stage-2 extractor recovers fields at 0.83–0.99 over 4,800 real NLLB translations in six languages (closed categoricals $\geq$ 0.97; open entity/relational fields 0.83–0.92), and the meaning-digest authenticates documents cross-lingually (core6 document attribution 1.000 for German/Hindi, 0.90–0.98 for the rest; Secs. 10.2, 10.4). The full run ships in-repo (`paper/results/realmt_b2/`); wide-coverage parsing and larger document counts remain open.
Wide-coverage neural parsing on open-domain text	Pilot (positive on open domain)	In a small LLM-MT pilot (96 sentences, six conditions) a neural extractor beats the lexicon on open-domain wording (0.614 vs. 0.481 field recovery) while the lexicon leads on closed templates; single-model and small-scale caveats apply (Sec. 10.3).
Benign paraphrase robustness	Demonstrated (closed-domain)	On 200 documents $\times$16 sentences, meaning-preserving in-lexicon paraphrase (voice, time reflow, verb/time synonyms, discourse hedge) authenticates at TPR 1.000 and document attribution 1.000; single-field altering edits rejected 1.000 (Sec. 10.7).
Adaptive-attack robustness	Closed lexicon refuted; defended by extended coverage; open-vocabulary open	A schema-aware attacker swapping out-of-lexicon content synonyms ("took care of" for "fixed") reaches ValidRemoval 1.000 (Eq. 1) under the closed lexicon, localized to the stripped predicate/time/attribution fields (polarity and agent survive 100%). An extended-coverage frontend recovers those fields, driving adaptive ValidRemoval to 0.000 while benign auth (1.000) and tamper rejection (1.000) are unchanged; a held-out out-of-inventory attack still evades any fixed lexicon (ValidRemoval 1.000), but two real open-vocabulary neural frontends (GPT-4o-mini, Llama-3.3-70B) defend even the held-out attack (ValidRemoval 0.050/0.100, CIs disjoint from the lexicons') at benign/tamper 1.000. Coverage, not the meaning-digest, is the lever; the neural frontend supplies open-vocabulary coverage (Sec. 10.7, Table 9).
System-level false-positive rate	Not established	The tag bound is per tested fresh candidate; parser search, tested key–nonce pairs, and fallback scores require end-to-end calibration.
Semantic-level	Measured	The $2^{-\tau}$ bound covers cryptographic tag collision

Table 8: Positioning of Truthprint relative to representative watermarking and attack research.

Category	Representative work	Main carrier or focus	Limitation addressed by Truthprint
Token-level sampling	SynthID-Text	Keyed token-selection statistics	Translation and paraphrasing replace token decisions.
Sentence-space watermarking	SemStamp	Keyed sentence-embedding regions	Embedding regions are not typed by invariant category.
Semantic-context watermarking	SIR, SemaMark	Contextual semantic embeddings	Similarity does not explicitly protect negation, numbers, causality, or attribution.
Hybrid defense	SynGuard	Token watermark plus semantic alignment	Semantic constraints are less explicit than a typed IR contract.
Structured semantics	SWAN	AMR-level structures	Existing semantic form is not a complete watermark-specific mutability contract.
Dual embeddings	DEW	Context and token embeddings	Language invariance remains implicit rather than represented as a canonical IR.
Attack analysis	Layer inflation, black-box probing, adversarial paraphrasing	Detector assumptions and adaptive transformation	Motivates authenticated payloads, conservative interfaces, and erasure-aware decoding.
Proposed system	Truthprint	Typed invariants and controlled realization classes	Explicitly specifies what must remain fixed and what may carry symbols.

Table 9: Open-vocabulary neural frontend vs. fixed lexicons on the adaptive corpus (contract core6, cached real-LLM extractions; Wilson 95% CI). *Two* instruction LLMs defend the *held-out novel* attack that both fixed lexicons fail, at unchanged benign auth and tamper rejection. N = documents scored per frontend.

Frontend	Benign TPR	Adaptive-seen VR	Held-out novel VR	Tamper rej.
Closed lexicon (N=40)	1.000	1.000	1.000	1.000
Extended lexicon (N=40)	1.000	0.000	1.000	1.000
Neural: GPT-4o-mini (N=40)	1.000	0.050	0.050	1.000
Neural: Llama-3.3-70B (N=10)	1.000	0.200	0.100	1.000

Table 10: Real head-to-head on real NLLB translation. A real KGW token watermark collapses under translation while the meaning-digest scheme survives. KGW is at a calibrated 1% FPR; the meaning-digest row is at its contract-collision operating point ($\sim 5 \times 10^{-3}$), so the two rows are not at a matched FPR and the clean cell (—) is undefined for a ledger scheme with no in-text signal. Section 10.5 (Table 1) gives the matched-FPR comparison, all methods at 1% FPR, adding a real semantic baseline and robust hashing. Replaces the diagnostic simulator (Appendix F).

Method	clean	RT	KO	HI	ZH	AR	DE
KGW (token)	0.960	0.380	0.040	0.040	0.020	0.000	0.080
Truthprint (meaning)	—	0.941	0.958	0.980	0.922	0.502	0.948

Table 11: Ledger retrieval(+NLI) vs. the typed meaning-digest, same ledger and id-addressed verification, benign-paraphrase acceptance matched at ≈ 0.95 (960 closed-domain items, Wilson 95% CI). Storing and retrieving the source survives paraphrase but cannot reject a single-field tamper; only a perfect (unrealizable) NLI matches the typed contract, and even then does not localize the field or give a keyed tag.

Method	Benign TPR	Tamper rej.	Localizes	Model
Meaning-digest (typed)	1.000	1.000	yes	none
Retrieval (NN)	0.941	0.047	no	encoder
Retrieval + NLI (proxy)	0.918	0.061	no	NLI
Retrieval + NLI (oracle)	0.941	1.000	no	perfect NLI

Table 12: Paraphrase, adaptive attack, and extended-lexicon defense on real English strings (200 documents $\times$16 sentences, contract core6, Wilson 95% CI). Benign meaning-preserving paraphrase is authenticated (RQ4); a schema-aware adaptive attacker (RQ7) removes the mark under the *closed* lexicon (ValidRemoval 1.000), but an *extended-coverage* frontend recovers the same invariants and drives ValidRemoval to 0.000 while still rejecting altering edits; a held-out attack whose synonyms lie outside the extended inventory still evades (coverage bound). Auth TPR is over meaning-preserving paraphrases; tamper rejection for the altering row.

Transform family	Auth TPR	ValidRemoval	Surface carrier
Benign paraphrase	1.000	0.000	0.015
Adaptive (RQ7), closed lexicon	0.000	1.000	0.000
Adaptive (RQ7), extended	1.000	0.000	0.000
Held-out novel, extended	0.000	1.000	0.000
Altering (tamper), extended	0.000	— (1.000 rej.)	0.000